Implementing Decision Trees
A report on Project 1
INF264

Ankita Johar

September 2026

Contents

1 Decision Tree Implementation **2**
1.1 The ID3 Algorithm 2
1.2 The Gini Index 3
1.3 Maximum Depth 4

2 Evaluating your Implementations **5**
2.1 Model Selection 5
2.2 Model Evaluation and Comparison to Existing Implementations 6

3 Feature Importance **8**

1 Decision Tree Implementation

1.1 The ID3 Algorithm

For my decision tree implementation, I created a `Node` class and a `DecisionTree` class. The `Node` class stores the information needed at each node in the tree. This includes the feature, threshold, and the left and right child. If the node is a leaf, the predicted class will be stored. I chose to keep the nodes separate from the main `DecisionTree` class because it made the tree structure easier to build and follow.

I built the tree recursively in the `build_tree()` method. I used recursion because after the data is split, the same process needs to be repeated for both the left and right subsets. Before finding a new split, I first check if the tree should stop growing. If all samples have the same label, the node is already pure and I create a leaf with that label. If all feature values are identical, it's not possible to split the data further. So I create a leaf using the majority class instead.

```
# stop if all data points have the same label
if len(np.unique(y)) == 1:
    return Node(value=y[0])

# stop if all data points have identical feature values
if np.all(X == X[0]):
    return Node(value=self.majority_class(y))
```

If the node doesn't meet one of the stopping conditions, I use the `best_split()` method to find the next split. The method goes through each feature and uses the median value of that feature as the threshold. I chose the median because it was one of the suggested approaches in the task and gives a simple way of creating a threshold for continuous features. Samples with values less than or equal to the threshold are placed on the left, while values above the threshold are placed on the right. I also check that both sides contain samples before using the split. If one side is empty, the split is skipped.

```
# use median value of the feature as the threshold
threshold = np.median(X[:, feature])

# split data into left and right groups
left_mask = X[:, feature] <= threshold
right_mask = X[:, feature] > threshold

# skip if the feature cannot split the data
if not np.any(left_mask) or not np.any(right_mask):
    continue
```

For every valid split, I calculate the information gain using the `information_gain()` method. I first calculate the impurity before the split. The impurity is then calculated for the left and right subsets and weighted based on the number of samples in each subset. I subtract this weighted impurity from the original impurity to get the information gain. A larger information gain means that the split has reduced the impurity more.

In `best_split()`, I compare the information gain for the different features. If a split has a higher gain than the best one found so far, I save its feature and threshold. After all features have been checked, the feature with the highest information gain is returned as the best split. This is how my implementation chooses the best available split at each node.

```
gain = self.information_gain(y, y_left, y_right)

# update if this split has the highest information gain
if gain > best_gain:
    best_gain = gain
    best_feature = feature
    best_threshold = threshold
```

I split both `X` and `y` into left and right subsets after the best feature and threshold has been found. I then call `build_tree()` again for each subset to create the left and right child. The feature, threshold and the two children are stored in a new `Node`. This process continues until a stopping condition is reached. If no valid split can be found, I also create a leaf using the majority class.

The `fit()` method starts this process by calling `build_tree()` with the training data and a starting depth of 0. Once the tree has been built, `predict()` is used to predict the labels of new samples. For each sample, `predict_one()` starts at the root and compares the selected feature value with the threshold. A value less than or equal to the threshold follows the left child, while a larger value follows the right child. This continues until a leaf is reached, where the stored class is returned as the prediction.

1.2 The Gini Index

In addition to entropy, I implemented the gini index as an alternative impurity measure. The impurity measure is selected using the `criterion` parameter in the `DecisionTree` constructor. The default value is `"entropy"`, but one can select `"gini"`.

I implemented both measures inside the same `impurity()` method. The method first finds the number of samples belonging to each class and then uses these counts to calculate the class probabilities. It then calculates either entropy or gini depending on the selected criterion.

```
# calculate entropy or gini index
def impurity(self, y):

# find how many samples belong to each class
    _, counts = np.unique(y, return_counts=True)
    probabilities = counts / len(y)

# calculate impurity using the selected criterion
    if self.criterion == "entropy":
        return -np.sum(probabilities * np.log2(probabilities))
    elif self.criterion == "gini":
        return 1 - np.sum(probabilities ** 2)
    else:
        raise ValueError("criterion must be 'entropy' or 'gini'")
```

I chose to keep both calculations in the same method because the rest of the tree building process is the same for both criteria. The `information_gain()` method calls `impurity()` when evaluating a split. So changing the criterion automatically changes which impurity measure is used. This meant that I didn't need separate tree building methods for entropy and gini.

A gini value of 0 means that all samples in a node belong to the same class. A larger value indicates that the node contains a mixture of classes. The tree therefore still searches for splits that reduce impurity, but the way the impurity is calculated depends on the selected criterion.

1.3 Maximum Depth

I implemented `max_depth` to control how deep the decision tree can grow. The value is given when creating the `DecisionTree`. If `max_depth=None`, then no maximum depth is specified and the tree continues growing until one of the other stopping conditions is reached.

To keep track of the current depth, `build_tree()` receives a `depth` argument. The tree starts at depth 0. The depth is increased by one each time the method is called recursively for a child node. Before finding a new split, I check if the selected maximum depth has been reached. If it has, the node becomes a leaf and the majority class of the remaining samples is used as its prediction.

```
# stop if maximum depth has been reached
if self.max_depth is not None and depth >= self.max_depth:
    return Node(value=self.majority_class(y))
```

When creating the child nodes, the depth is increased before the next recursive call:

```
# recursively build the left and right subtrees
left_child = self.build_tree(X_left, y_left, depth + 1)
right_child = self.build_tree(X_right, y_right, depth + 1)
```

I included maximum depth as a stopping condition because it gives control over the size of the tree instead of always allowing it to grow until the other stopping conditions are reached. It also allowed me to compare different tree depths during model selection, and to select a suitable value based on validation performance.

2 Evaluating your Implementations

2.1 Model Selection

Before performing model selection, I split the dataset into 80% training data and 20% test data. This resulted in 5625 samples in the training set and 1407 samples in the test set. I chose an 80/20 split so that most of the data could be used for training and cross-validation, while still keeping a separate part of the dataset for the final evaluation. The test set was not used during model selection and remained unseen until the final model had been selected.

Before choosing the performance measure, I looked at the class distribution in the dataset. There were 5163 samples from class 0 (no churn) and 1869 samples from class 1 (churn). This showed that the dataset was imbalanced, with fewer samples belonging to the churn class. I therefore chose F1-score as the performance measure for model selection. A simple accuracy score would not give the full picture. It could favor a tree that only guessed the majority class every time. F1-score was more suitable because it takes both precision and recall into account.

The two hyperparameters I tested were `criterion` and `max_depth`. For `criterion`, I tested both `"entropy"` and `"gini"`, since these were the two impurity measures implemented in my decision tree. For `max_depth`, I tested maximum depths from 1 to 10 to compare trees with different levels of complexity without making the hyperparameter search unnecessarily large. I also included None to evaluate the tree without a specified maximum depth.

I used 5-fold cross-validation on the training set to evaluate each hyperparameter combination. The training data was divided into five folds. Four folds were used to train the model and the remaining fold was used for validation. This was repeated so that each fold was used for validation once. I used cross-validation instead of a single validation split because the result would depend more on which samples happened to be placed in the validation set. Using several validation folds gave a more reliable comparison between the different hyperparameter settings.

I chose five folds to balance between using enough data for training in each iteration, and keeping the computational cost manageable. For every combination of `criterion` and `max_depth`, I calculated the F1-score on each of the five validation folds and then calculated the mean of these scores. The hyperparameter combination with the highest mean cross-validation F1-score was selected.

To make the model selection fair, every hyperparameter combination was evaluated using the same cross-validation folds and the same F1-score metric. The test set was kept completely separate and was not used to choose any hyperparameters. I also used a fixed random seed of 42 for the train-test split, and when shuffling the training data for the cross-validation folds. This makes the data splits reproducible and allows it to be repeated using the same setup.

Figure 1 shows the mean cross-validation F1-score for the different maximum depths. The highest mean cross-validation F1-score was 0.6019 at a maximum depth of 2. Entropy and gini achieved the same score at this depth. Since entropy appeared first in the tied results when selecting the maximum score, entropy was selected. The final hyperparameters selected for my implementation were therefore `criterion="entropy"` and `max_depth=2`.

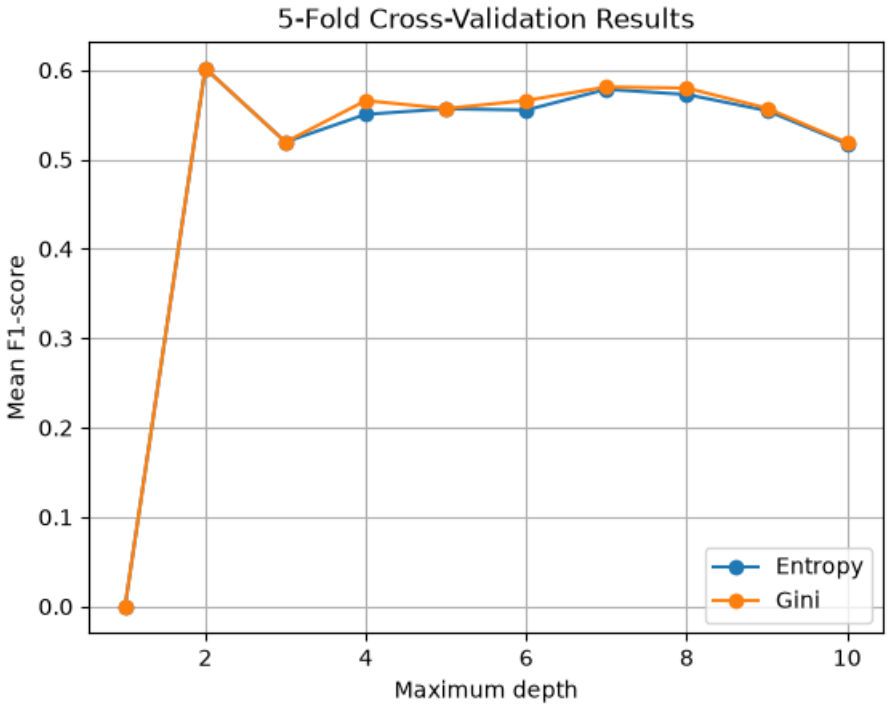


Figure 1: Mean F1-score for different maximum depths using 5-fold cross-validation.

2.2 Model Evaluation and Comparison to Existing Implementations

To compare my decision tree with an existing implementation, I used sklearn's `DecisionTreeClassifier`. I performed a separate model selection for the sklearn implementation. This was done to give both implementations the possibility to select the hyperparameters that performed best for them.

I used the same model selection setup for both implementations. The sklearn model was evaluated using the same training data, 5-fold cross-validation folds, F1-score, and hyperparameter values. I tested both entropy and gini for `criterion`, together with `max_depth` values from 1 to 10 and `None`. The test set was again kept separate from this process.

The best sklearn model achieved a mean cross-validation F1-score of 0.5921 with a maximum depth of 2. Entropy and gini achieved the same score at this depth. Similar to my implementation, entropy appeared first in the tied results and was therefore selected. The selected sklearn model used `criterion="entropy"` and `max_depth=2`.

After model selection was completed, I created a new model from each implementation using its selected hyperparameters. Each model was then trained on the full training set. This allowed the models to use all of the training data, after the cross-validation process had been used to select the hyperparameters. The models were then evaluated once on the test set, which had not been used during model selection. I used F1-score for the final evaluation so that the final results could be compared using the same performance measure that was used during model selection.

Table 1: Comparison of the selected decision tree models.

Model	Criterion	Max depth	Mean CV F1	Test F1
Own implementation	Entropy	2	0.6019	0.5758
Sklearn	Entropy	2	0.5921	0.5758

As shown in Table 1, both implementations selected entropy and a maximum depth of 2. My implementation achieved a slightly higher mean cross-validation F1-score of 0.6019 compared with 0.5921 for sklearn. However, this difference was not present in the final evaluation, where both models achieved a test F1-score of 0.5758.

The test F1-score was slightly lower than the mean cross-validation F1-score for both models. My implementation decreased from a mean cross-validation F1-score of 0.6019 to a test F1-score of 0.5758, while the sklearn implementation decreased from 0.5921 to 0.5758. Some difference between validation and test performance can be expected because the models are evaluated on different samples.

An F1-score of 0.5758 represents the balance between precision and recall for the churn class. Since the score is not close to 1, the models are still making errors when identifying customers who churn. However, the purpose of the comparison is mainly to evaluate how my implementation performs compared with an existing implementation under the same conditions.

My implementation had a slightly higher mean cross-validation F1-score than sklearn, but this did not continue on the unseen test set. Both models achieved exactly the same test F1-score of 0.5758. This suggests that the small difference during cross-validation was not enough to result in a difference in final test performance. Based on the test F1-score, my implementation therefore achieved comparable performance to sklearn's decision tree on this dataset.

3 Feature Importance

I implemented permutation importance to examine which features had the largest influence on the predictions made by my decision tree. I used my own selected decision tree from the previous section, with entropy and a maximum depth of 2. The model was trained on the full training set, while the permutation importance was calculated on the test set. I used accuracy as the performance measure, 30 permutations for each feature, and a fixed random seed of 42, as stated in the task.

I implemented the permutation importance algorithm. The baseline accuracy was first calculated on the unchanged test set. For each feature, its values were then randomly shuffled and the accuracy was calculated again. This was repeated 30 times for each feature, and the importance score was calculated as the difference between the baseline accuracy and the mean accuracy after permutation. A larger decrease in accuracy therefore gives a larger importance score.

The results are shown in Figure 2. Contract had the highest permutation importance with a score of 0.0475, followed by OnlineSecurity with 0.0403. The remaining features had importance scores close to 0.

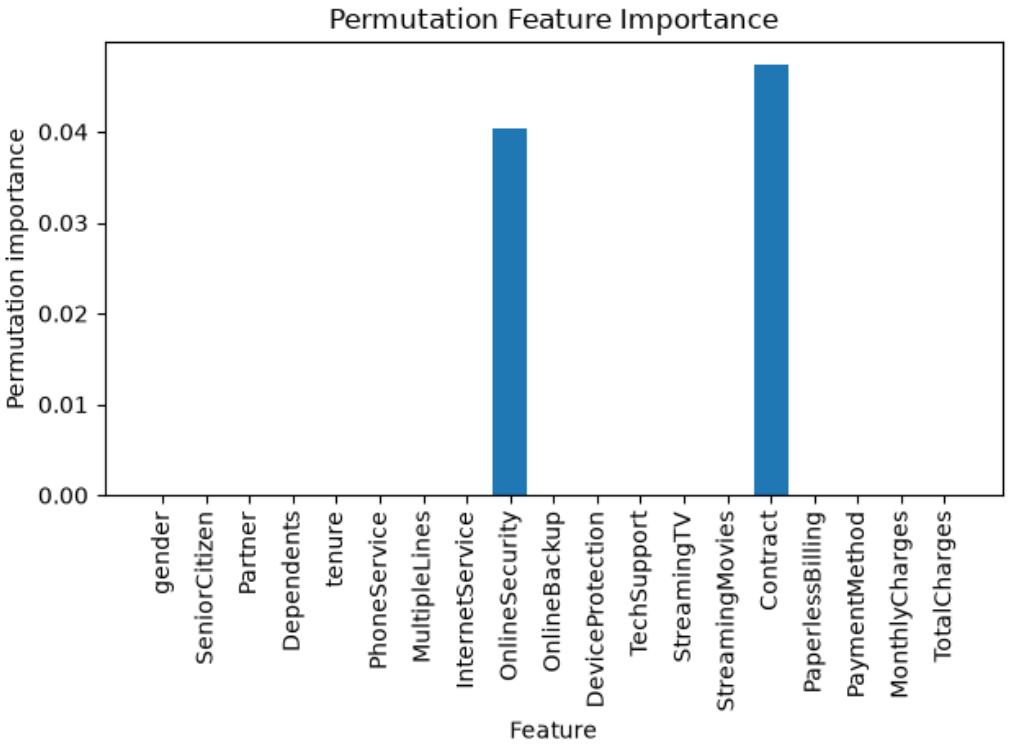


Figure 2: Permutation importance of each feature using accuracy on the test set.

The results show that the model's accuracy decreased the most when Contract was shuffled. An importance score of 0.0475 means that the model's accuracy dropped by about 0.0475 on average when the feature was shuffled. OnlineSecurity also caused a noticeable decrease in accuracy when shuffled. This suggests that the predictions depended more on Contract and OnlineSecurity than on the other features.

Features with importance scores close to zero should be interpreted carefully. A score close to zero does not mean that the feature is unrelated to customer churn. It only means that shuffling the feature did not have much effect on the model's accuracy. This could be because the model does not use the feature much, or because another feature provides similar information.

One weakness of permutation importance is therefore that the result depends on the trained model. It does not measure how important a feature is in general or if the feature causes customer churn. A different model could use the available features differently and therefore produce different importance scores.

Another weakness occurs when features are related or contain similar information. If two features provide similar information, the model may still be able to make good predictions using one feature after the other has been shuffled. This can make an important feature appear less important than it actually is.

Randomly shuffling a feature can also break the relationship it has with other features and create combinations that would not normally appear in the dataset. This means the model may be tested on less realistic data. The scores can also change depending on the test data, the performance metric, and the random shuffling. Repeating the process 30 times makes the result more reliable, but does not completely remove these weaknesses.

The permutation importance results show which features this specific decision tree relied on for its predictions. They should not be interpreted as a general ranking of which factors are most important for customer churn.